

Abstract

- HESTIA is a real estate buyer self-service Android application that helps users easily explore, search, and manage properties based on their requirements.
- Built using Kotlin (Android), Node.js microservices with REST APIs, and MySQL for data storage.
- Provides a centralized platform with property listings, detailed information, images, EMI calculator, saved properties and user interest management.

Problem Statement & Objectives

- Problem:** Existing real estate platforms are complex, cluttered and not focused on first-time home buyers, making it difficult to find suitable properties and get key information quickly.
- Objective 1:** Provide an easy-to-use mobile application for property search and exploration.
- Objective 2:** Offer detailed property information with images, floor plans and location details.
- Objective 3:** Enable user features like save/shortlist, interest management and EMI calculation.
- Scope:** Android app for buyers, backend services for data management, limited to residential properties initially.

Existing vs Proposed System

Feature	Existing System	Proposed System (HESTIA)
User Experience	Complex, cluttered UI	Clean, simple and mobile-friendly UI
Property Information	Limited details	Detailed info with images, floor plans and location
Search & Filters	Basic search	Advanced search and filters (budget, type, location, etc.)
Save / Shortlist	Not available or limited	Save favourite properties
EMI Calculation	Not available	Inbuilt EMI calculator
User Interest Management	Not available	Register interest / inquiry
Notifications	Mostly email based	Real-time notifications (FCM)
Platform	Web only	Android mobile application

Feasibility Study

	Technical Feasibility	- Uses modern and widely supported technologies (Android, Node.js, MySQL) - Tools and platforms are easily available
	Economic Feasibility	- Open-source stack (no licensing cost) - Can be deployed using free-tier cloud services
	Operational Feasibility	- User-friendly interface, easy to use - Simple maintenance with modular microservices
	Schedule Feasibility	- Achievable within the semester timeline - Work distributed among team members

Results & Testing

- Successfully implemented all core features (CRUD operations)
- API testing done using Postman
- Properties are fetched from MySQL database (24 seeded records)
- User authentication and JWT-based authorization working
- EMI calculator producing correct results
- Application tested on real Android device (end-to-end flow)
- All modules working as expected

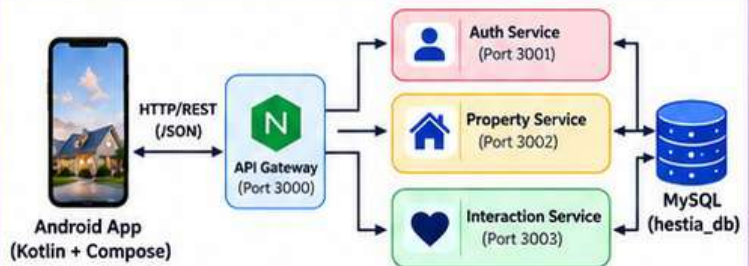
Conclusion

- Developed a fully functional Android application for real estate property search and management.
- Integrated multiple backend services with MySQL database using REST APIs.
- Provides a user-friendly platform for property discovery with useful features like EMI calculator, save/shortlist and interest management.
- Meets all the stated objectives successfully.

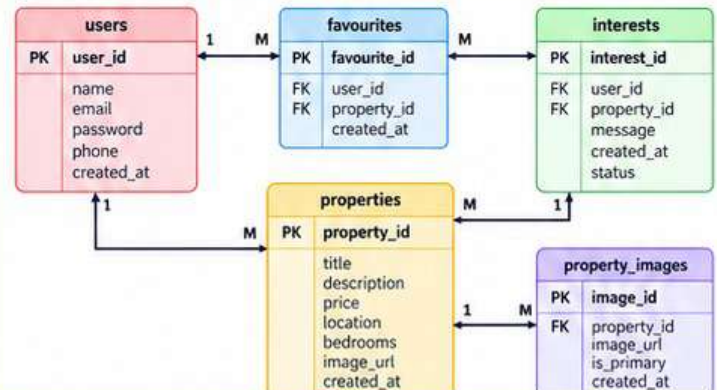
Tools & Technologies

	Front-End	: Kotlin, Android Studio, Jetpack Compose
	Back-End	: Node.js (Microservices) + REST APIs
	Database	: MySQL (hestia_db)
	Tools	: Git & GitHub, Postman, VS Code, Draw.io
	Others	: Firebase Cloud Messaging (Notifications)

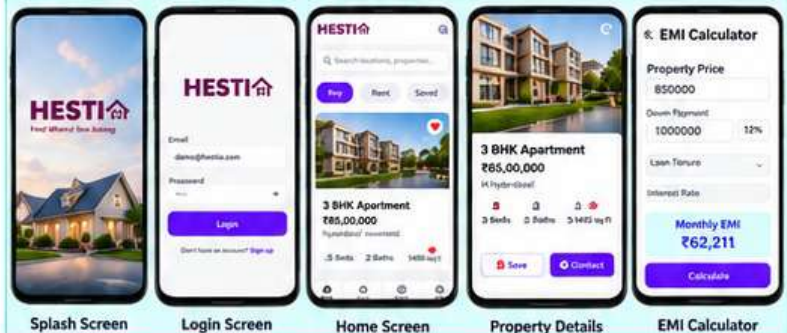
System Architecture



ER Diagram & Database Design



Implementation Screenshots



Future Scope

- Add map-based property search and location visualization
- Integrate advanced recommendation system using ML
- Add chat feature between buyers and property agents
- Support for property owner/agent registration
- Deploy on cloud for public access
- Add virtual tours (360° view) and enhanced filters

